

Permutation Equivariant Model-based Offline Reinforcement Learning for Auto-bidding

Zhiyu Mou
mouzhiyu.mzy@taobao.com
Alibaba Group
Beijing, China

Miao Xu
xumiao.xm@taobao.com
Alibaba Group
Beijing, China

Wei Chen
ganhai.cw@taobao.com
Alibaba Group
Beijing, China

Rongquan Bai
rongquan.br@taobao.com
Alibaba Group
Beijing, China

Chuan Yu
yuchuan.yc@alibaba-inc.com
Alibaba Group
Beijing, China

Jian Xu
xiyu.xj@alibaba-inc.com
Alibaba Group
Beijing, China

ABSTRACT

Recently, reinforcement learning (RL)-based auto-bidding technique has experienced a paradigm shift: from training policies with the *imaginary data* generated by a simplistic offline simulator of the online advertising system, which we term the *Simulation-based RL Bidding* (SRLB), to training policies with a fixed set of *real data* collected directly from the online advertising system with offline RL methods, which we term the *Offline RL Bidding* (ORLB). However, the trajectories of the policies trained by the ORLB are often restricted to the vicinity of the real data, and as the real data typically covers a limited state space, the trained policy exhibits only modest performance improvements. Although the imaginary data in the SRLB can expand the covered state space, it comes with a risk: it can seriously mislead the policy due to the significant gap between the simplistic offline simulator and the online advertising system. In this paper, we introduce a *Model-based RL Bidding* (MRLB) paradigm that holds promise for addressing the challenges present in the SRLB and the ORLB. Specifically, the MRLB first learns an environment model through supervised learning based on real data to serve as the offline simulator, which can bridge the gap encountered in the SRLB. Then the MRLB trains the policy with both real and imaginary data, which can expand the covered state space compared to that of the ORLB. As the performance of the MRLB heavily relies on the accuracy of the environment model, we propose two novel designs to further improve the reliability of the environment model for policy training. Firstly, we design the environment model as a *permutation equivariant* neural network to enhance its generalization ability with theoretical guarantees. Secondly, we design a robust offline Q learning method that pessimistically penalizes the reward and state predictions of the environment model to alleviate the negative impact of potential overestimations on policy training. These two designs constitute a specific algorithm based on the MRLB, named *Permutation Equivariant Model-based Offline*

RL (PE-MORL). Extensive real-world experiments validate the superiority of the proposed PE-MORL algorithm compared to the state-of-the-art auto-bidding algorithms.

CCS CONCEPTS

• Information systems → Computational advertising; • Theory of computation → Reinforcement learning.

KEYWORDS

Auto-bidding, Reinforcement Learning, Permutation Equivariance

ACM Reference Format:

Zhiyu Mou, Miao Xu, Wei Chen, Rongquan Bai, Chuan Yu, and Jian Xu. 2018. Permutation Equivariant Model-based Offline Reinforcement Learning for Auto-bidding. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 14 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 INTRODUCTION

Auto-bidding has become a popular tool to bid for impression opportunities on behalf of advertisers in online advertising systems. As auto-bidding inherently involves the sequential decision-making problem, reinforcement learning (RL) [23] is playing an increasingly important role in acquiring outstanding auto-bidding policies. In recent years, the RL-based auto-bidding technique has experienced a paradigm shift in policy training. The initial paradigm mainly focuses on training auto-bidding policies in a simplistic, manually-constructed offline simulator of the online advertising system, which we term the *Simulation-based RL Bidding* (SRLB) paradigm [7–9, 25, 26, 29], since directly training in the online advertising system is typically not allowed due to safety concerns. However, due to the commonly existing gap between the offline simulator and the online advertising system, the data generated by the offline simulator — referred to as the *imaginary data* — is typically biased, potentially resulting in sub-optimal or even poor auto-bidding policies in the online advertising system. This is known as the *inconsistencies between online and offline* (IBOO) problem [19]. To tackle this problem, the current focus has shifted away from training policies with the offline simulator to directly using the fixed dataset collected from the online advertising system — referred to as the *real data* — to train the auto-bidding policy with model-free offline RL methods, which we term the *Model-free Offline RL Bidding* (ORLB) paradigm [1, 12, 14, 19]. However, the

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference acronym 'XX, June 03–05, 2018, Woodstock, NY

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/18/06
<https://doi.org/XXXXXXX.XXXXXXX>

trajectories of the auto-bidding policy trained by the ORLB are often constrained to the vicinity of the real data, typically covering a limited state space [19], which only results in modest improvements in the performance of the auto-bidding policy.

Essentially, both paradigms fail to fully exploit the information that could be useful for training the auto-bidding policy. Specifically, (1) for ORLB: the information loss occurs in the portions of the state space that are not covered by the real data. Actually, this missing information can be supplemented by generating additional data through further interactions with an offline simulator; (2) for SRLB: although the imaginary data generated by the offline simulator can expand the covered state space, existing offline simulators are generally constructed with the simplistic General Second Principle (GSP) rule. This approach overlooks much of the useful and available information about the bidding environment that could be contained in the real data (or could be derived from professional expertise), and as previously mentioned, these offline simulators exhibit serious IBOO problems. In fact, we can employ more effective methods to construct the offline simulator with more useful information, which can largely mitigate the IBOO problem.

Based on these analyses, here we introduce the *Model-based RL Bidding* (MRLB) paradigm to the auto-bidding field, as described in Algorithm 1. The MRLB paradigm is mainly based on the single-agent model-based RL algorithm, which has been proven to be effective in many other tasks, such as robotics [20] and video games [10]. Specifically, the MRLB first learns an environment model through supervised learning based on real data, which serves as the offline simulator. This can increase the amount of information contained in the offline simulator compared to the GSP-based offline simulator in the SRLB and largely mitigate the IBOO problem. Then the MRLB trains the auto-bidding policy with both the real data and the imaginary data, which expands the covered state space compared to that of the ORLB. Hence, the MRLB acts as a promising paradigm to compensate for the information losses in both the SRLB and the ORLB.

However, the vanilla MRLB algorithm that directly follows Algorithm 1 without any particular designs is still not enough to train outstanding auto-bidding policies due to the prohibition of additional interactions with online advertising systems during training and the fixed nature of the real data. Specifically, we note that the performance of the MRLB heavily relies on the accuracy of the environment model. Since the fixed real data typically covers a limited state space [19], the environment model may be inaccurate for states outside the real data. Such inaccuracies, particularly in overestimating rewards, can mislead the policy and significantly impair the policy's performance. This is typically known as the *Out-Of-Distribution* (OOD) challenge [12, 19, 27].

In this paper, we propose a specific algorithm of the MRLB paradigm, named the *Permutation Equivariant Model-based Offline RL* (PE-MORL), to address the OOD challenge. The PE-MORL features two novel designs compared to the vanilla MRLB algorithm.

- Firstly, we design the environment model as a *permutation equivariant* model, which can have better generalization ability and accuracy both theoretically and empirically. Specifically, rather than only using the features of a single agent (i.e., advertiser in this paper) as inputs, which is a typical

Algorithm 1 MRLB Paradigm

Input: Real data $\mathcal{D}_R$

Output: Auto-bidding policy $\hat{\pi}$

- 1: Initialize the imaginary data $\mathcal{D}_I \leftarrow \emptyset$ and the auto-bidding policy $\hat{\pi}$;
 - 2: Learn an environment model $\hat{M}$ based on $\mathcal{D}_R$ with supervised learning;
 - 3: **repeat**
 - 4: Interact with $\hat{M}$ using $\hat{\pi}$ to generate imaginary data and store it in $\mathcal{D}_I$;
 - 5: Update $\hat{\pi}$ with $\mathcal{D}_R \cup \mathcal{D}_I$;
 - 6: **until** The expected return of $\hat{\pi}$ in $\hat{M}$ converges.
-

design in single-agent model-based RL algorithms, our environment model incorporates both the states and actions of the considered advertiser and those of all the competing advertisers as inputs, and outputs the predictions on the next states of all advertisers and the reward of the considered advertiser. Note that the incorporation of all advertisers' information can naturally enhance the generalization ability of the environment model. The "*permutation equivariant*" means the property that if the order of advertisers in the inputs to the environment model changes, then the output values will remain the same but their orders will permute accordingly. An insight behind this design is that the order of inputs does not alter the information they contain, and therefore, only the order of the output values should change accordingly [15]. Theoretically, we can prove that making the environment model a permutation equivariant function can increase its generalization ability.

- Secondly, we design a robust offline Q learning method to deal with the uncertainty of the states and actions the environment predicts, which can further address the OOD challenge. Specifically, in RL algorithms, the vanilla Q loss function is only a Bellman error term (as introduced in Section 3), which can be easily influenced by the overestimation of the environment model, making the trained auto-bidding policy improperly deviate from the real data. In the proposed robust offline Q learning method, we take the minimum Q value of the states predicted by the environment model and penalize the predicted reward based on the uncertainty of the environment model to robustly deal with the overestimations.

Furthermore, extensive simulated and real-world experiments validate the superiority of the proposed PE-MORL algorithm compared with the state-of-the-art auto-bidding policies. Particularly, we validate that the designed environment model exhibits improved generalization ability compared to a conventional neural network, which is not permutation equivariant, and to the GSP-based offline simulator commonly used in SRLB.

2 RELATED WORKS

We briefly review the existing algorithms based on the MRLB paradigms in the following, and the complete related works, including the mean-field game algorithms, offline RL algorithms, and permutation equivariance RL algorithms, are given in Appendix A.

Model-based RL Bidding (MRLB): The design of algorithms based on the MRLB paradigm can date back to [2]. However, it attempts to model the environment at the granularity of individual impression opportunities, which is impractical in online advertising systems. Very recently, [3] proposed an algorithm based on the MRLB, named Model-Based Automatic Bidding (MBAB). However, MBAB resembles the vanilla MRLB algorithm, which can perform poorly when the real data is fixed as stated in Section 1. Notably, the proposed PE-MORL can address the OOD challenge caused by the fixed real data and is practical in online advertising systems.

3 NOTATIONS AND PRELIMINARIES

Key Notations: $[N]$ denotes the positive integer set containing 1 to N , where $N \in \mathbb{N}_+$; $[\cdot]_+ \triangleq \max\{\cdot, 0\}$; $|\mathcal{D}|$ represents the number of elements in the set $\mathcal{D}$; $\circ$ is the function composition operation; $\|\cdot\|_1$ denotes the 1-norm; $\cup$ denotes the union operator between sets, and $\setminus$ denotes the subtraction operator between sets;

In the following, we define some important concepts, operators, and properties that will be used later.

Definition 3.1 (Ordered Vector). An ordered vector, denoted as $\mathbf{x} \triangleq x_{1:N} \triangleq [x_1, x_2, \dots, x_N]^\top$, is a vector with elements arranged in a specific sequence, where x_i is the i -th element, $i \in [N]$.

Definition 3.2 (Permutation Operator). A permutation operator ρ is a function that rearranges the elements of an ordered vector into a different sequence.

Let Ω_N be the set of permutation operators operating on N -dimensional ordered vectors. Note that there are $N!$ distinct permutation operators in Ω_N . We next define the permutation equivariance and permutation invariance concepts that will play key roles in our later design of the environment model.

Definition 3.3 (Permutation Equivariant and Permutation Invariant). A function $f : \mathcal{X} \rightarrow \mathcal{Y}$ is permutation equivariant (PE) if it holds that $f(\rho\mathbf{x}) = \rho\mathbf{y}$, $\forall \rho$, where $\mathbf{x} \in \mathcal{X}$ and $\mathbf{y} = f(\mathbf{x}) \in \mathcal{Y}$ are ordered vectors. Moreover, a function $f : \mathcal{X} \rightarrow \mathbb{R}$ is permutation invariant (PI) if it holds that $f(\rho\mathbf{x}) = y$, $\forall \rho$, where $y = f(\mathbf{x})$.

3.1 Single-agent Reinforcement Learning

Single-agent RL [23] focuses on learning optimal sequential decisions for an agent by interacting with an environment. The decision-making processes is typically modeled as a *Partially Observed Markov Decision Process* (POMDP) [13] $\langle \mathcal{S}, \mathcal{A}, \mathcal{O}, \Omega, R, P, \gamma \rangle$. At each time step $t \in \mathbb{N}_+$, $s_t \in \mathcal{S}$ is the state of the agent at time step t , $o_t \in \mathcal{O}$ is the observation, $a_t \in \mathcal{A}$ is the operation performed by the agent to interact with the environment. Then R determines the immediate reward r_t to the agent, and the agent's next state s_{t+1} are transitioned by P . The agent's task is to find a policy $a_t = \pi(s_t)$ that can maximize the cumulative rewards. Besides, $\gamma \in [0, 1]$ is the discounted factor.

The key to most RL methods lies in learning an optimal Q function. The Q function, denoted as $Q^\pi(s_t, a_t)$, represents the expected

cumulative reward from taking action a_t in the state s_t and following a certain policy hereinafter. To evaluate the Q function of the current policy, a common approach is to train it with the Mean Squared Error (MSE) loss between the current Q value and the target Q value, i.e.,

$$\mathcal{L} = \mathbb{E}_{(s_t, a_t, r_t, s_{t+1}) \sim D} \left[(Q^\pi(s_t, a_t) - \hat{B}Q^\pi(s_t, a_t))^2 \right], \quad (1)$$

where

$$\hat{B}Q^\pi(s_t, a_t) = r_t(s_t, a_t) + \gamma \max_{a_{t+1}} Q^\pi(s_{t+1}, a_{t+1}) \quad (2)$$

is the target Q value, computed by the Bellman operator.

4 PROBLEM SETTINGS

In online advertising, the bidding process typically involves $N (> 1)$ advertisers competing for impression opportunities simultaneously with a certain policy each, such as the auto-bidding policy. Here, we focus on learning the optimal auto-bidding policy of a single advertiser, referred to as the *representative advertiser*, under the condition where the policies of other $N - 1$ advertisers, referred to as the *background advertisers*, are fixed. Specifically, the bidding process of the representative advertiser can be modeled as a POMDP, $\langle \mathcal{S}, \mathcal{A}, \mathcal{O}, \Omega, R, P, \gamma \rangle$, with $T \in \mathbb{N}_+$ time steps, where $s_t \in \mathcal{S}$ denotes the global state of all advertisers at time step $t \in [T]$, and $a_{N,t} \in \mathcal{A}$ denotes the local action, i.e., the bid, of the representative advertiser at time step t . Due to privacy concerns, the representative advertisers can only obtain an observation $o_t \in \mathcal{O}$ which does not contain any information about other advertisers and is also called as local state $s_{N,t}$. In practice, o_t contains individual information of advertiser (e.g. the constraint and budget of advertiser, the category and sales history of their product, etc.), the current status of advertiser (e.g. time left, budget left, budget spend speed, etc.) and the prediction of the ad slot valuations, etc. Besides, We denote the action of background advertiser i at time step t as $a_{i,t}$, and the observation as $s_{i,t}$. The space of these two variables are also denoted as $\mathcal{A}$ and $\mathcal{O}$. Then, the global state can be considered as the concatenation of all local states $s_{1:N,t}$. And let the joint action of all advertisers at time step t be $\mathbf{a}_t \triangleq a_{1:N,t} \in \mathcal{A}^N$, where $\mathcal{A}^N$ denotes the joint action space.

Hence, the policy¹ of the representative advertiser should be constructed as a mapping from the observation space to the probabilistic space of the action space, i.e., $\pi : \mathcal{O} \rightarrow \Delta(\mathcal{A})$. Moreover, $R : \mathcal{S} \times \mathcal{A}^N \rightarrow \mathbb{R}$ is the reward function of the representative advertiser, and $P : \mathcal{S} \times \mathcal{A}^N \rightarrow \Delta(\mathcal{S})$ is the transition rule. The bidding process can be described as follows: at each time step t , the representative advertiser takes action $a_{N,t} \sim \pi(o_t)$ and each background advertiser i takes action $a_{i,t}$ at the same time; then the representative advertiser receives the total value of impression opportunities won by her or him as the reward $r_t = R(s_t, \mathbf{a}_t)$ and pays the corresponding cost c_t ; afterward, all the advertisers transition to the next state s_{t+1} according to the transition rule P . Besides, $\gamma = 1$ is used in our system. The goal of the considered auto-bidding policy is to maximize the expected accumulative rewards of the

¹For convenience, we refer to the auto-bidding policy simply as the "policy".

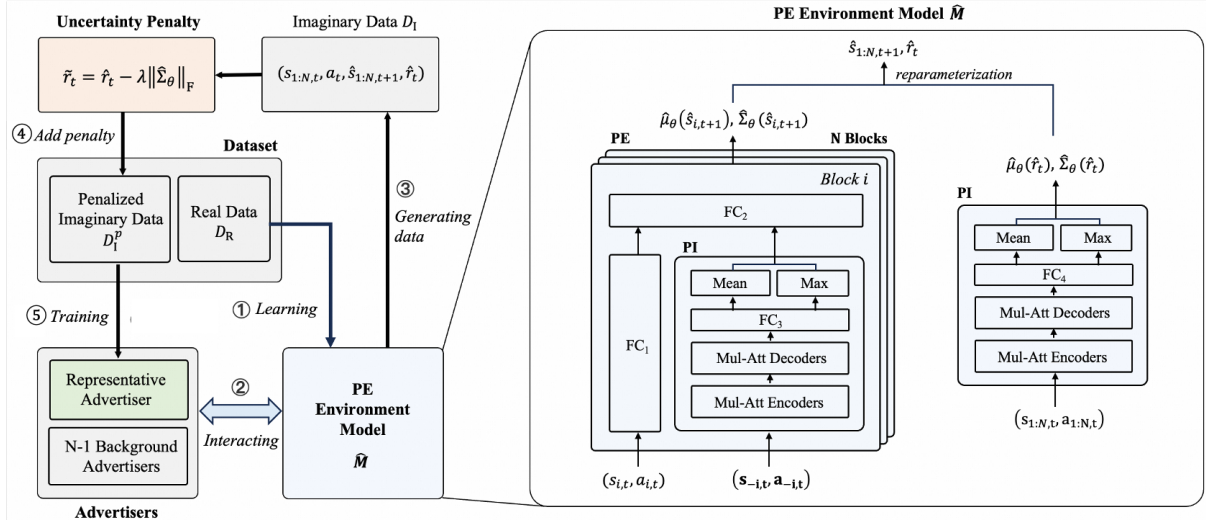


Figure 1: The proposed PE-MORL algorithm is designed based on the MBRL paradigm. The PE-MORL follows the MBRL paradigm: it first learns an environment model and then trains the auto-bidding policy of the representative advertiser with the environment model and the real data. Moreover, the PE-MORL features two novel designs. The first design is the PE environment model with better generalization ability, whose architecture is shown on the right. The second design is the uncertainty penalty to avoid the OOD challenge.

representative agent under the budget constraint, i.e.,

$$\max_{\pi} \mathbb{E}_{\pi, P} \left[\sum_{t=1}^T \gamma^t r_t \right], \quad \text{s.t.} \quad \sum_{t=1}^T c_t \leq B, \quad (3)$$

where $B > 0$ denotes the budget of the representative advertiser. We next propose an MRLB algorithm, named PE-MORL, to solve (3).

5 ALGORITHM

Fig. 1 shows the proposed PE-MORL algorithm. Compared with the vanilla MBRL paradigm described in Algorithm. 1, the PE-MORL features two novel designs. Firstly, we design the environment model as a PE function to improve its generalization ability with theoretical guarantees. Then, to further reduce the negative impact of the inaccuracies of the environment model, especially the overestimations, on policy training, we design a robust offline Q learning method that can enhance the reliability of the imaginary data. In the following, we dive into the details of these two designs and summarize the whole algorithm in Algorithm 2.

5.1 Permutation Equivariant Environment Model

As the environment model inherently models the reward function R and the transition rule P — both are functions of the global state space $\mathcal{S}$ and the joint action space $\mathcal{A}^N$, we design the environment model as a function $\hat{M}$ that outputs the predictions on the next global state $\hat{s}_{t+1}$ and the reward $\hat{r}_t$ with the global state s_t and the joint action a_t as inputs. Note that although the representative advertiser cannot get s_t and a_t during the online inference, we can

obtain them during the offline training² and use them to construct the environment model $\hat{M}$. Moreover, we can prove that the reward function R is a PI function and the transition rule P is a PE function in the following proposition.

PROPOSITION 5.1 (THE PE AND PI PROPERTIES OF THE POMDP). *In the POMDP of the representation advertiser, the transition rule P is a PE function, i.e., $\rho s_{t+1} \sim P(\rho s_t, \rho a_t)$, and the reward R is a PI function, i.e., $R(\rho s_t, \rho a_t) = R(s_t, a_t)$, $\forall t, \rho$.*

In essence, the reason lies in the PE and PI property of the auction mechanism in the online advertising system concerning the local states and the bids of all advertisers. The detailed proof is given in Appendix E.1. Hence, we design the environment model $\hat{M}$ to be a PE function from the perspective of $\hat{s}_{t+1}$ and a PI function from the perspective of $\hat{r}_t$, i.e., for any permutation operator ρ , we have:

$$\rho \hat{s}_{t+1}, \hat{r}_t = \hat{M}(\rho s_t, \rho a_t). \quad (4)$$

We name the designed environment model $\hat{M}$ as the *PE environment model*³. A direct insight behind this design is that the information contained in s_t and a_t does not change no matter how we swap the elements in them, and hence, $\hat{r}_t$ and the values of the elements in $\hat{s}_{t+1}$ should not be changed, only the positions of the elements in $\hat{s}_{t+1}$ should be swapped accordingly [16]. Moreover, as we know one of the most desirable characteristics of the environment models is their strong ability for generalization.

We evaluate the generalization ability of an environment model M by the gap between its accuracy on the training dataset $\mathcal{S}$, and

²Note that the policy of the representative advertiser π can still only take the observation o_t as input during the offline training.

³Note that as (4) can be equivalently expressed as (29), we directly name the environment model as the PE environment model.

its accuracy on the test dataset $\mathcal{D}$, i.e.,

$$\Delta_G(\hat{M}) \triangleq \mathbb{E}_{\mathcal{D}}[L(\hat{M}, \mathbf{s}_t, \mathbf{a}_t)] - \frac{1}{|\mathcal{S}|} \sum_{\mathcal{S}} L(\hat{M}, \mathbf{s}_t, \mathbf{a}_t), \quad (5)$$

where $L(\hat{M}, \mathbf{s}_t, \mathbf{a}_t)$ is the loss function (here we use the mean absolute error (MAE) as the loss function) and can represent the accuracy of the environment model. Note that a smaller gap $\Delta_G(\hat{M})$ indicates a better generalization ability of the environment model M since we can reduce the loss of the environment model on the training dataset to obtain a lower value of the loss of the environment model on the test dataset, which indicates a better generalization ability. Notably, we can prove that the design in (4) can reduce the upper bound of the gap $\Delta_G(\hat{M})$.

THEOREM 5.2 (BETTER GENERALIZATION ABILITY OF THE PE ENVIRONMENT MODEL). *For any environment model $\hat{M}'$, making it satisfy (4) can reduce the upper bound of the gap $\Delta_G(\hat{M}')$, i.e.,*

$$\text{Upper Bound of } \Delta_G(\hat{M}) \leq \text{Upper Bound of } \Delta_G(\hat{M}'), \quad (6)$$

where $\hat{M}$ denotes the environment model that satisfies (4).

The proof is given in Appendix D. Therefore, the environment model with the design of (4) can have a better generalization ability.

5.1.1 Architecture Design: To design an environment model $\hat{M}$ that satisfies (4), a straightforward way is to randomly build a function approximator, e.g. a neural network, and apply the orbit averaging $\mathcal{Q}$ to it, as is described previously. However, as the number of background advertisers is usually large and applying the orbit averaging requires $N!$ times of calculations every time, it is time infeasible in practice. Hence, we here design a neural network as the environment model $\hat{M}$ with the structure that naturally satisfies (4). We next dive into the details of its structure.

Remark 1. Intuitively, a neural network that naturally satisfies (4) can be expected to easily approximate the functions after orbit averaging $\mathcal{Q}$ than a vanilla neural network does, especially when the training dataset is not sufficient [5, 21].

As shown in Fig. 1, the environment model $\hat{M}$ is mainly composed of two neural networks, where the left one is a PE function, responsible for predicting the next state $\hat{\mathbf{s}}_t$ and the right one is a PI function, responsible for predicting the reward $\hat{r}_t$.

The Left Neural Network contains N blocks with identical structures and shared parameters, where block i takes the global state $\mathbf{s}_t$ and the joint action $\mathbf{a}_t$ as inputs and generates the prediction on the local state, $\hat{\mathbf{s}}_{i,t+1}$. Specifically, block i is composed of three modules, including (1) a fully connected neural network, named FC_1 , used to extract the embedding of the corresponding advertiser; (2) a PI neural network used to extract the embedding of all other advertisers; and (3) a fully connected neural network, named FC_2 , used to deal with the embeddings generated by the first two modules and output the predictions $\hat{\mathbf{s}}_{i,t+1}$. The insight of designing a PI neural network for feature extractions of all other agents is that: the input order of other agents can neither change the information contained in $(\mathbf{s}_{-i,t}, \mathbf{a}_{-i,t})$ nor influence the situation the corresponding advertiser faces. Notably, the structure of the PI neural network is designed based on the following lemma.

LEMMA 5.3 (THEOREM 2 IN [28]). *All PI functions can be represented as $\phi(\text{pool}(\psi(\cdot)))$ when $\text{pool}(\cdot)$ is the sum operator and $\phi(\cdot)$ and $\psi(\cdot)$ are continuous.*

The PI neural network is designed in the form of $\phi(\text{pool}(\psi(\cdot)))$. Specifically, we use several multi-head attention (Mul-Att) encoders and decoders [24] as $\psi(\cdot)$. Note that there is a fully connected neural network at the end of $\psi(\cdot)$ across attention heads to extract features between different heads. In block i , we denote the output of $\psi(\cdot)$ as $\mathbf{B}_{i,t} \triangleq [\mathbf{b}_{0,t}^i, \dots, \mathbf{b}_{i-1,t}^i, \mathbf{b}_{i+1,t}^i, \dots, \mathbf{b}_{N,t}^i]$, where $\mathbf{b}_{j,t}^i \in \mathbb{R}^B$ is the extracted feature for the advertiser with $(s_{j,t}, a_{j,t})$, where $j \in [N] \cup \{0\} \setminus \{i\}$. For the $\text{pool}(\cdot)$ function, we use two functions, including a mean function that is inherently the same as the sum operator (as required in Lemma 5.3), and a max function. The insight for using the max function is that: for each advertiser, usually only the highest bid of other advertisers will influence its reward and state transition. Let $\mathbf{m}_t^i \triangleq \mathbf{m}_{1:N,t}^i \in \mathbb{R}^B$ and $\mathbf{g}_t^i \triangleq \mathbf{g}_{1:N,t}^i \in \mathbb{R}^B$ be the outputs of the mean function and the max function, respectively. Then we have $\forall k \in [B]$:

$$\mathbf{m}_{k,t}^i = \frac{1}{N} \sum_{j \in [N] \cup \{0\} \setminus \{i\}} b_{j,t}^{i,k}, \quad (7)$$

where $b_{j,t}^{i,k}$ denotes the k -th element in $\mathbf{b}_{j,t}^i$, and

$$\mathbf{g}_{k,t}^i = \max_j \left\{ b_{j,t}^{i,k} \mid j \in [N] \cup \{0\} \setminus \{i\} \right\}. \quad (8)$$

Afterward, $\mathbf{m}_t^i$ and $\mathbf{g}_t^i$ are concatenated and processed by FC_2 together with the outputs of FC_1 . Here FC_2 can be viewed as the function $\phi(\cdot)$ in Lemma 5.3. Note that we assume that the ground truth $\mathbf{s}_{i,t+1}$ obeys a certain Gaussian distribution. Hence, FC_2 is designed to generate the mean value of $\hat{\mathbf{s}}_{i,t+1}$, denoted as $\hat{\mu}_\theta(\hat{\mathbf{s}}_{i,t+1})$, and its variance and covariances with the predictions on all other next local states and the reward, denoted as $\hat{\Sigma}_\theta(\hat{\mathbf{s}}_{i,t+1})$, where θ represents the trainable parameters of all the environment model $\hat{M}$. As different blocks have the same structure and share the parameters, all the N blocks constitute a PE function.

The Right Neural Network contains only a single block with the same architecture as the PI neural network on the left but with independent parameters and different input dimensions. Specifically, the Mul-Att encoder takes $(\mathbf{s}_t, \mathbf{a}_t)$ as inputs rather than $(\mathbf{s}_{-i,t}, \mathbf{a}_{-i,t})$. The right neural network generates the predictions on the mean of $\hat{r}_t$, denoted as $\hat{\mu}_\theta(\hat{r}_t)$ as well as its variance and covariances with the predictions on the next local states, denoted as $\hat{\Sigma}_\theta(\hat{r}_t)$.

The Outputs of the left and right neural networks constitute the predictions on the mean, denoted as $\hat{\mu}_\theta$, and the covariance matrix, denoted as $\hat{\Sigma}_\theta$, of all the next local states and the reward. Specifically, the mean $\hat{\mu}_\theta$ can be represented as:

$$\hat{\mu}_\theta \triangleq [\hat{\mu}_\theta(\hat{\mathbf{s}}_{1,t+1}), \dots, \hat{\mu}_\theta(\hat{\mathbf{s}}_{N,t+1}), \hat{\mu}_\theta(\hat{r}_t)]. \quad (9)$$

To guarantee the symmetric property of the covariance matrix, the covariance matrix $\hat{\Sigma}_\theta$ is designed as:

$$\hat{\Sigma}_\theta = \frac{1}{2} (\tilde{\Sigma}_\theta + \tilde{\Sigma}_\theta^\top), \quad (10)$$

where $\tilde{\Sigma}_\theta$ is defined as:

$$\tilde{\Sigma}_\theta \triangleq [\hat{\Sigma}_\theta(\hat{\mathbf{s}}_{1,t+1}), \dots, \hat{\Sigma}_\theta(\hat{\mathbf{s}}_{N,t+1}), \hat{\Sigma}_\theta(\hat{r}_t)]^\top. \quad (11)$$

Then, the predicted next local state $\hat{s}_{t+1}$ and the reward $\hat{r}_t$ are generated by the re-parameterization method [18] based on $\hat{\mu}_\theta$ and $\hat{\Sigma}_\theta$, i.e.,

$$\begin{bmatrix} \hat{s}_{t+1} \\ \hat{r}_t \end{bmatrix} = \hat{\mu}_\theta + \mathbf{L}\epsilon, \quad (12)$$

where $\mathbf{L}$ is the Cholesky factorization of the covariance matrix, i.e., $\hat{\Sigma}_\theta = \mathbf{L}\mathbf{L}^\top$, and $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ is a random vector obeying the standard Gaussian distribution.

The Loss Function is designed as the maximum log-likelihood function with regularization terms [11], i.e.,

$$\mathcal{L}(\theta) = \mathbb{E}_{[s_{t+1}, r_t]^\top \sim \mathcal{D}_R} \left[\left(\begin{bmatrix} s_{t+1} \\ r_t \end{bmatrix} - \hat{\mu}_\theta \right)^\top \Sigma_\theta^{-1} \left(\begin{bmatrix} s_{t+1} \\ r_t \end{bmatrix} - \hat{\mu}_\theta \right) + \log |\hat{\Sigma}_\theta| + \|\hat{\Sigma}_\theta\|_F \right], \quad (13)$$

where $\mathcal{D}_R = \{(s_t, \mathbf{a}_t, r_t, s_{t+1})\}$ denotes the real data collected from the online advertising system, containing the real transition pairs. To train the equivariant environment model $\hat{M}$, we apply the gradient descent method based on $\mathcal{L}(\theta)$, i.e.,

$$\theta \leftarrow \theta - \alpha \nabla_\theta \mathcal{L}(\theta), \quad (14)$$

where $\alpha > 0$ is the update step.

Ensemble Environment Models: To increase the robustness of data generated by the environment model, we train an ensemble of K environment models. During the training phase, the auto-bidding policy interacts with all K environment models, and we randomly pick the prediction of an environment model as the final prediction and store it into the imaginary data, denoted as $\mathcal{D}_I$. Specifically, the imaginary data consists of the state transition pairs of the representative advertiser, i.e., $\mathcal{D}_I = \{(o_t, a_{N,t}, \hat{r}_t, \hat{o}_{t+1})\}$, where $\hat{o}_{t+1} = O(\hat{s}_{t+1})$.

5.2 Robust Offline Q Learning Method

Given that the real data $\mathcal{D}_R$ is fixed, the environment model is not subject to iterative enhancement concurrent with the policy training. Hence, the inaccuracies of the environment model at certain states outside the real data $\mathcal{D}_R$ will persist and may seriously mislead the training of the policy. Particularly, the environment model's overestimations of rewards can seriously jeopardize the policy since it might incorrectly deviate from the original data $\mathcal{D}_R$, leading to a significant performance decline. In addition, if the states predicted by the environment model have erroneous large Q values, then it can also mislead the policy since they act as the target Q value in the Q loss function as described in (1).

Our main idea to address these challenges is to pessimistically treat the predicted rewards of the environment model and the Q values of its predicted states. Specifically, in the robust offline Q learning method, we modify the target Q value $\hat{B}Q(s_t, a_t)$ in the loss function of Q as:

$$\hat{B}Q(s_t, a_t) = \underbrace{\hat{r}_t - \lambda \|\hat{\sigma}_t\|_F}_{\text{reward penalty}} + \gamma \underbrace{\min_{\hat{s}_{t+1} \in \hat{M}(s, a)} (\max_{a' \in \mathcal{A}} Q(\hat{s}_{t+1}, a'))}_{\text{state penalty}}, \quad (15)$$

where $\hat{M}(s, a)$ denotes the set of the samples of the environment model's prediction on states, $\hat{\sigma}_t$ denotes the variance of the ensemble environment model's predicted rewards, and $\lambda > 0$ is a constant.

In the state penalty, we take the minimum of the max Q value of the predicted state to treat the predictions of the environment model pessimistically. This can increase the robustness of the training policy even in the worst case. As for the reward penalty, we note that the environment model's inaccuracy is higher in states outside the real data $\mathcal{D}_R$ and lower in states within it, which is consistent with the value of $\hat{\sigma}_t$ [22, 27]. Therefore, penalizing it can make the policy more pessimistic about states outside the real data $\mathcal{D}_R$, thereby decreasing the risk of deviating incorrectly from the real data $\mathcal{D}_R$. Intuitively, the penalized predicted reward $\tilde{r}_t \triangleq \hat{r}_t - \lambda \|\hat{\sigma}_t\|_F$ could have lower values as the state-action pairs move away from the real data $\mathcal{D}_R$, which can constrain the trajectories of the policy within the proximity of $\mathcal{D}_R$. Theoretically, we can prove that optimizing the policy in the environment model $\hat{M}$ with penalized predicted reward $\tilde{r}_t$ can improve the lower bound of the policy's performance in the online advertising system M .

PROPOSITION 5.4 (LOWER BOUND IMPROVEMENT). *For any policy π , its performance in the online advertising system M , denoted as $\eta_M(\pi)$, is lower bounded by its performance in a fitted environment model $\hat{M}$ with the penalized predicted reward $\tilde{r}_t$, denoted as $\eta_{\hat{M}}(\pi)$, i.e., $\eta_M(\pi) \geq \eta_{\hat{M}}(\pi)$.*

The proof is given in Appendix E.4, where we leverage the proof technique in [27].

Penalized Imaginary Data: After adding the penalty to the predicted reward $\hat{r}_t$ in the imaginary data, we further obtain the penalized imaginary data $\mathcal{D}_I^p = \{(s_t, \mathbf{a}_t, \tilde{r}_t, \hat{s}_{t+1})\}$. The penalized imaginary data $\mathcal{D}_I^p$ and the real data $\mathcal{D}_R$ together constitute the training dataset used for policy training.

Policy Training: We leverage an actor-critic framework in training the policy, where the Q function and the policy are updated iteratively [6]. The loss function of the policy is its negative return, which is exactly $-Q^{\pi_w}$, i.e.,

$$l(w) = \mathbb{E}_{\mathcal{D}_R \cup \mathcal{D}_I^p} \left[-Q^{\pi_w}(o_t, a_{N,t}) \right]. \quad (16)$$

where w denotes the parameter of the policy. We use the gradient descent to update w , i.e.,

$$w \leftarrow w - \eta \nabla_w l(w), \quad (17)$$

where $\eta > 0$ is the step size.

5.3 Overall Algorithm

The overall algorithm is described in Algorithm 2. Specifically, we first learn a PE environment model $\hat{M}$ based on the real data $\mathcal{D}_R$. Then we train the Q function with the target (15) and learn the policy of the representation advertiser with (16) based on the imaginary data and the real data $\mathcal{D}_R \cup \mathcal{D}_I^p$. Specifically, at each iteration, we interact with the trained equivariant environment model $\hat{M}$ using the policy π_w ⁴. The imaginary data $(s_t, \mathbf{a}_t, \hat{r}_t, \hat{s}_{t+1})$ will be generated during the interactions. After penalizing the reward, we store $(s_t, \mathbf{a}_t, \tilde{r}_t, \hat{s}_{t+1})$ into the penalized imaginary data $\mathcal{D}_I^p$. With $\mathcal{D}_R \cup \mathcal{D}_I^p$, we update the policy parameter w based on (17). Update

⁴Note that we assume the policies of all the background advertisers are known in advance.

Algorithm 2 PE-MORL**Input:** The real data $\mathcal{D}_R$, hyper-parameters $\lambda, \alpha, \beta, \eta$;**Output:** π_w (that can be viewed as the solution to (3));

- 1: Initialize the penalized imaginary data $\mathcal{D}_I^P \leftarrow \emptyset$ and the policy parameter w ;
- 2: Learn an equivariant environment model $\hat{M}$ based on the real data $\mathcal{D}_R$ with (14);
- 3: **repeat**
- 4: Interact with $\hat{M}$ using π_w to generate the imaginary data $(s_t, a_t, \hat{r}_t, \hat{s}_{t+1})$;
- 5: Obtain the penalized predicted reward $\tilde{r}_t$ and store $(s_t, a_t, \tilde{r}_t, \hat{s}_{t+1})$ in $\mathcal{D}_I^P$;
- 6: Update the Q function Q^{π_w} with loss function (1), where the target Q value leverages (15).
- 7: Update π_w with $\mathcal{D}_R \cup \mathcal{D}_I^P$ based on (17);
- 8: **until** The expected return of π_w in $\hat{M}$ converges.

the policy parameter w until the return of π_w in $\hat{M}$ converges. Then we output π_w as an approximate solution to (3).

6 EXPERIMENTS

We conduct extensive real-world experiments to validate the effectiveness of our approach. We mainly study the following four questions in the experiments: (1) What is the overall performance of the PE-MORL algorithm compared to the state-of-the-art auto-bidding algorithms? (2) What is the performance of the equivariant environment model compared to the commonly used bidding environment? Does the equivariance design help with its generalization ability? (3) Can the proposed model-based approach, PE-MORL, truly enable policies to transcend the limitations imposed by the real data $\mathcal{D}_R$ compared with the ORLB algorithm? (4) Does the uncertainty penalty work?

Experiment Setup. We conduct real-world experiments on the online advertising system of one of the world’s largest E-commerce platforms, TaoBao. We leverage A/B Testing to evaluate our approach on thousands of representative advertisers each with thousands of background advertisers with fixed local policies. An episode is set to be one day. There are $T = 48$ time steps, and the duration between two-time steps is half an hour.

Performance Metric. The main performance metric in our experiments is the objective function, which represents the total value of impression opportunities won by the advertiser in an episode. This metric is subsequently referred to as the ‘GMV’ throughout the following sections. Besides, we use other two indexes that are commonly used in the auto-bidding problem to evaluate the performance of the policy. The first metric is the total consumed budget (Cost) of the advertiser. The second metric is the return on investment (ROI) which is defined as the ratio between the GMV and the Cost of the advertiser. Note that larger values of GMV, ROI, and no significant decrease in Cost indicate better performance of the auto-bidding policy. As for the environment models, we utilize the mean square error (MSE) and the mean absolute error (MAE) to evaluate their performance.

Baselines. We compare the proposed PE-MORL with the state-of-the-art auto-bidding algorithms, including the algorithm based

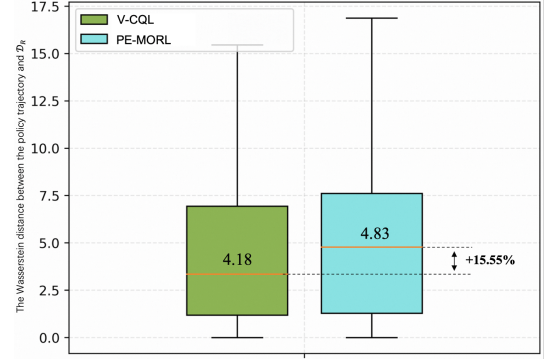


Figure 2: The Wasserstein distance between the policy trajectories and the real data $\mathcal{D}_R$.

on the SRLB: USCB [8] that trains policies in a commonly used GSP-based offline simulator, the algorithm based on the ORLB: V-CQL⁵ [19], a model-free offline RL algorithm; and a vanilla MRLB algorithm: MBAB [3]. In the ablation study, to validate the effectiveness of the proposed equivariant environment model $\hat{M}$, we compare it with the GSP-based offline simulator and a neural network-based environment model without the design of (4), constructed by only fully connected layers.

6.1 Real-World Experiments

To Answer Question (1): We conduct A/B Testing on the real-world online advertising system between the proposed PE-MORL and the state-of-the-art baselines for thousands of representative advertisers over multiple days. The results are shown in Table 1. We can see that the GMV of the PE-MORL significantly exceeds those in the V-CQL, USCB, and MBAB. Besides, nearly all other performance metrics, including ROI and Cost of the PE-MORL exceed those in the baselines. These validate the superiority of the proposed PE-MORL.

To Answer Question (2): We first compare the proposed equivariant environment model $\hat{M}$ with a neural network-based model without the design of (4). Both models are trained on real data collected from the online advertising system in 7 days, and evaluated on another real data. The results are given in Table 2. We can see that the test loss of $\hat{M}$ is lower than that of the environment model without the design of (4). Moreover, we compare $\hat{M}$ with the GSP-based offline simulator that is commonly used in the SRLB, and the results are given in Table 3. We can see that the equivariant environment model $\hat{M}$ can significantly improve the prediction accuracy on both the next state and reward compared to the GSP-based offline simulator.

To Answer Question (3): We compare the distance between the real data $\mathcal{D}_R$ and the policy trajectories trained by the ORLB algorithm, V-CQL, denoted as d_{V-CQL} , and the PE-MORL algorithm, denoted as $d_{PE-MORL}$, respectively. The distance is calculated by the

⁵V-CQL is the SORL proposed by [19] without online explorations. Note that we focus on the offline algorithms and the online exploration is out of scope.

Table 1: A/B Testing in the real-world experiments: comparing V-CQL, USCB, and MBAB with PE-MORL: 3k representative advertisers lasting for 10 days.

Algorithms	GMV	ROI	Cost	Algorithms	GMV	ROI	Cost	Algorithms	GMV	ROI	Cost
V-CQL	6,313,011	4.50	1,402,298	USCB	6,059,062	5.09	1,190,859	MBAB	7,200,474	4.57	1,574,216
PE-MORL (Ours)	6,591,894	4.68	1,409,574	PE-MORL (Ours)	6,497,634	5.41	1,201,845	PE-MORL (Ours)	7,482,550	4.83	1,548,506
Diff	+4.4%	+3.9%	+0.5%	Diff	+7.2%	+6.3%	+0.9%	Diff	+3.9%	+5.7%	-1.6%

Table 2: The test MAE and MSE losses of the environment model constructed by the fully connected neural network without the design of (4) and the designed PE environment model.

Test Loss	Fully-connected Environment Model without (4)	PE Environment Model $\hat{M}$	Diff
MAE	0.404	0.381	-5.9%
MSE	1.469	1.420	-3.3%

Table 3: The test MAE and MSE losses of the GSP-based offline simulator and the designed PE environment model.

Test Loss	GSP-based Offline Simulator	PE Environment Model $\hat{M}$	Diff
MAE	1.253	0.381	-69.6%
MSE	12.034	1.420	-88.2%

Wasserstein metric, and the results are given in Fig. 2. We can see that the average value of $d_{\text{PE-MORL}}$ is 15.55% higher than $d_{\text{V-CQL}}$, which indicates that the PE-MORL can indeed break through the real data $\mathcal{D}_R$ to find better policies. **Visualizations** of the policy trajectories are given in Fig. 3, and the corresponding performances of each single representation advertiser are given in Table. 4. We can see that the trajectories of policies trained by the PE-MORL are further from the real data and exhibit better performance.

6.2 Ablation Study

To Answer Question (4): To validate the effectiveness of the uncertainty penalty design, we evaluate the policy under the different values of λ , and the results are given in Table. 5. Note that R/R^* and online rate are commonly used offline metrics in the auto-bidding [8]. Lower R/R^* and online rate indicate the poorer performance of the policy. We can see that a smaller value of λ can lead to poor policy performance. When $\lambda = 0$, there is no uncertainty penalty in the PE-MORL, and the trained policy performs the worst. This validates the effectiveness of the uncertainty penalty method.

7 CONCLUSIONS

In this paper, we analyze a recent paradigm shift in the RL-based auto-bidding technique: from the SRLB to the ORLB and point out the problems existing in both paradigms. We then introduce the MRLB, a promising paradigm to solve the problems in both the SRLB and the ORLB. However, the vanilla MRLB algorithm may

Table 4: A/B Testing in the real-world experiments between V-CQL and PE-MORL: 2 representative advertisers lasting for 5 days.

Representation Advertiser	Algorithms	GMV	ROI	Cost
Left in Fig. 3	V-CQL	16,940	2.25	7,543
	PE-MORL (Ours)	23,995	3.26	7,365
	Diff	+41.7%	+45.1%	-2.4%
Right in Fig. 3	V-CQL	241	1.37	175
	PE-MORL (Ours)	502	2.70	186
	Diff	+108.5%	+96.8%	+6.0%

Table 5: Ablation study under different λ .

λ	R/R^*	Online Rate	λ	R/R^*	Online Rate
5	0.8786	0.834	2	0.7714	0.783
4	0.8788	0.822	1	0.6493	0.401
3	0.8831	0.835	0	0.4519	0.412

still have poor performance due to the OOD challenge caused by the fixed real data. Hence, we propose a specific algorithm based on the MRLB, named the PE-MORL, that can further address the OOD challenge. Compared with the vanilla algorithm derived by the MRLB, the PE-MORL features two novel designs. Firstly, we design the environment model as a PE neural network with theoretical guarantees to increase its generalization ability. Secondly, we design a penalty method on the predicted reward of the environment model. The penalty method can alleviate the negative impact on policy training caused by overestimating the predicted rewards. Extensive real-world experiments validate the superiority of the proposed PE-MORL algorithm compared with the state-of-the-art auto-bidding policies. Particularly, we also validate that the designed environment model has better generalization ability compared to a vanilla neural network that is not permutation equivariant and the GSP-based offline simulator commonly used in the SRLB.

ACKNOWLEDGMENTS

To Robert, for the bagels and explaining CMYK and color spaces.

REFERENCES

- [1] Nair Ashvin, Dalal Murtaza, Gupta Abhishek, and I Sergey. 2020. Accelerating online reinforcement learning with offline datasets. *CoRR*, vol. *abs/2006.09359*

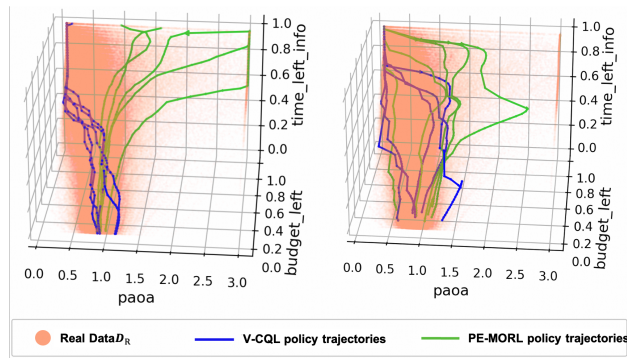


Figure 3: Policy trajectories of two specific representative advertisers trained by V-CQL and PE-MORL algorithms.

- (2020).
- [2] Han Cai, Kan Ren, Weinan Zhang, Kleanthis Malialis, Jun Wang, Yong Yu, and Defeng Guo. 2017. Real-time bidding by reinforcement learning in display advertising. In *Proceedings of the tenth ACM international conference on web search and data mining*. 661–670.
 - [3] Shuang Chen, Qisen Xu, Liang Zhang, Yongbo Jin, Wenhao Li, and Linjian Mo. 2023. Model-Based Reinforcement Learning for Auto-bidding in Display Advertising. In *Proceedings of the 2023 International Conference on Autonomous Agents and Multiagent Systems*. 1560–1568.
 - [4] Zhijian Duan, Yunxuan Ma, and Xiaotie Deng. 2023. Are Equivariant Equilibrium Approximators Beneficial? *arXiv preprint arXiv:2301.11481* (2023).
 - [5] Zhijian Duan, Jingwu Tang, Yutong Yin, Zhe Feng, Xiang Yan, Manzil Zaheer, and Xiaotie Deng. 2022. A context-integrated transformer-based neural network for auction design. In *International Conference on Machine Learning*. PMLR, 5609–5626.
 - [6] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. 2018. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *International conference on machine learning*. PMLR, 1861–1870.
 - [7] Xiaotian Hao, Zhaoqing Peng, Yi Ma, Guan Wang, Junqi Jin, Jianye Hao, Shan Chen, Rongquan Bai, Mingzhou Xie, Miao Xu, et al. 2020. Dynamic knapsack optimization towards efficient multi-channel sequential advertising. In *International Conference on Machine Learning*. PMLR, 4060–4070.
 - [8] Yue He, Xiujun Chen, Di Wu, Junwei Pan, Qing Tan, Chuan Yu, Jian Xu, and Xiaoqiang Zhu. 2021. A unified solution to constrained bidding in online display advertising. In *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*. 2993–3001.
 - [9] Junqi Jin, Chengru Song, Han Li, Kun Gai, Jun Wang, and Weinan Zhang. 2018. Real-time bidding with multi-agent reinforcement learning in display advertising. In *Proceedings of the 27th ACM international conference on information and knowledge management*. 2193–2201.
 - [10] Lukasz Kaiser, Mohammad Babaeizadeh, Piotr Milos, Blazej Osinski, Roy H Campbell, Konrad Czechowski, Dumitru Erhan, Chelsea Finn, Piotr Kozakowski, Sergey Levine, et al. 2019. Model-based reinforcement learning for atari. *arXiv preprint arXiv:1903.00374* (2019).
 - [11] Alex Kendall and Yarin Gal. 2017. What uncertainties do we need in bayesian deep learning for computer vision? *Advances in neural information processing systems* 30 (2017).
 - [12] Dmytro Korenkevych, Frank Cheng, Artsiom Balakir, Alex Nikulkov, Lingnan Gao, Zhihao Cen, Zuobing Xu, and Zheqing Zhu. 2023. Offline reinforcement learning for optimizing production bidding policies. *arXiv preprint arXiv:2310.09426* (2023).
 - [13] Vikram Krishnamurthy. 2016. *Partially observed Markov decision processes*. Cambridge university press.
 - [14] Aviral Kumar, Aurick Zhou, George Tucker, and Sergey Levine. 2020. Conservative q-learning for offline reinforcement learning. *Advances in Neural Information Processing Systems* 33 (2020), 1179–1191.
 - [15] Juho Lee, Yoonho Lee, Jungtaek Kim, Adam Kosiorek, Seungjin Choi, and Yee Whye Teh. 2019. Set transformer: A framework for attention-based permutation-invariant neural networks. In *International conference on machine learning*. PMLR, 3744–3753.
 - [16] Yan Li, Lingxiao Wang, Jiachen Yang, Ethan Wang, Zhaoran Wang, Tuo Zhao, and Hongyuan Zha. 2021. Permutation invariant policy optimization for mean-field multi-agent reinforcement learning: A principled approach. *arXiv preprint arXiv:2105.08268* (2021).
 - [17] Yuping Luo, Huazhe Xu, Yuanzhi Li, Yuandong Tian, Trevor Darrell, and Tengyu Ma. 2018. Algorithmic framework for model-based deep reinforcement learning with theoretical guarantees. *arXiv preprint arXiv:1807.03858* (2018).
 - [18] Shakir Mohamed, Mihaela Rosca, Michael Figurnov, and Andriy Mnih. 2020. Monte carlo gradient estimation in machine learning. *The Journal of Machine Learning Research* 21, 1 (2020), 5183–5244.
 - [19] Zhiyu Mou, Yusen Huo, Rongquan Bai, Mingzhou Xie, Chuan Yu, Jian Xu, and Bo Zheng. 2022. Sustainable Online Reinforcement Learning for Auto-bidding. *Advances in Neural Information Processing Systems* 35 (2022), 2651–2663.
 - [20] Athanasios S Polydoros and Lazaros Nalpantidis. 2017. Survey of model-based reinforcement learning: Applications on robotics. *Journal of Intelligent & Robotic Systems* 86, 2 (2017), 153–173.
 - [21] Tian Qin, Fengxiang He, Dingfeng Shi, Wenbing Huang, and Dacheng Tao. 2022. Benefits of permutation-equivariance in auction mechanisms. *Advances in Neural Information Processing Systems* 35 (2022), 18131–18142.
 - [22] Eric Schulz, Maarten Speekenbrink, and Andreas Krause. 2018. A tutorial on Gaussian process regression: Modelling, exploring, and exploiting functions. *Journal of Mathematical Psychology* 85 (2018), 1–16.
 - [23] Richard S Sutton, David McAllester, Satinder Singh, and Yishay Mansour. 1999. Policy gradient methods for reinforcement learning with function approximation. *Advances in neural information processing systems* 12 (1999).
 - [24] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).
 - [25] Chao Wen, Miao Xu, Zhilin Zhang, Zhenzhe Zheng, Yuhui Wang, Xiangyu Liu, Yu Rong, Dong Xie, Xiaoyang Tan, Chuan Yu, et al. 2022. A cooperative-competitive multi-agent framework for auto-bidding in online advertising. In *Proceedings of the Fifteenth ACM International Conference on Web Search and Data Mining*. 1129–1139.
 - [26] Di Wu, Xiujun Chen, Xun Yang, Hao Wang, Qing Tan, Xiaoxun Zhang, Jian Xu, and Kun Gai. 2018. Budget constrained bidding by model-free reinforcement learning in display advertising. In *Proceedings of the 27th ACM International Conference on Information and Knowledge Management*. 1443–1451.
 - [27] Tianhe Yu, Garrett Thomas, Lantao Yu, Stefano Ermon, James Y Zou, Sergey Levine, Chelsea Finn, and Tengyu Ma. 2020. Mopo: Model-based offline policy optimization. *Advances in Neural Information Processing Systems* 33 (2020), 14129–14142.
 - [28] Manzil Zaheer, Satwik Kottur, Siamak Ravanbakhsh, Barnabas Poczos, Russ R Salakhutdinov, and Alexander J Smola. 2017. Deep sets. *Advances in neural information processing systems* 30 (2017).
 - [29] Jun Zhao, Guang Qiu, Ziyu Guan, Wei Zhao, and Xiaofei He. 2018. Deep reinforcement learning for sponsored search real-time bidding. In *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*. 1021–1030.

A RELATED WORKS

A.1 Mean Field Game Algorithms

Mean field game (MFG) theory studies decision-making for a large population of homogeneous agents with small interactions. The basic idea of the MFG is to model the game of all agents as a game of two players intertwined with each other, including a representative agent and the empirical distribution of all other agents. The MFG algorithms usually look for the optimal policy in terms of social welfare under cooperative settings [16?] or Nash Equilibrium policies under competitive/general-sum setting [?]. For example, [1] designs a pessimistic value iteration method under the offline setting to maximize social welfare, and [?] proposes a Q-learning-based algorithm with better convergence stabilities and learning accuracies. However, in this paper, we study the auto-bidding problem from the perspective of a single advertiser (representative advertiser) where other advertisers' (background advertisers) policies are assumed to be fixed and known. (Note that this assumption is viable in practice since the auto-bidding policy designer, usually the commercial department of an online advertising platform, can access the bidding policy of all advertisers, and there is no ethical issue since the observation of the considered advertiser's policy involves no information on other advertisers.)

A.2 Offline Reinforcement Learning

Offline RL (also known as batch RL) aims to learn better policies based on a fixed offline dataset collected by some behavior policies. The main challenge offline RL addressed is the extrapolation error (also known as the out-of-distribution, OOD) caused by missing data. The key technique to address the extrapolation error is to be pessimistic about the state-action pairs outside the offline dataset and conservatively train the policy. Similar to the traditional online RL algorithms, the offline RL algorithms can be divided into the model-free offline RL and the model-based offline RL.

Model-free Offline RL directly trains the policy with the offline dataset without building or learning an environment model. Based on the specific ways of addressing the extrapolation error, the model-free offline RL algorithms can be further divided into policy constraint methods such as BCQ [?], BEAR [?], and conservative regularization methods such as CQL [14], as well as the constrained stationary distribution methods such as AlgaeDICE [?] and OptiDICE [?]. Note that the CQL acts as a strong baseline and has been applied to real-world auto-bidding systems [10].

Model-based Offline RL, in contrast, first learns an environment model and then trains the policy with it. Note that model-free offline RL algorithms can only learn on the states in the offline dataset, leading to overly conservative algorithms. Nonetheless, the model-based offline RL algorithms have the potential for broader generalization by generating and training on additional imaginary data [?]. The model-based offline algorithms can be divided into two categories. The first category is to build a pessimistic environment model based on the uncertainty estimations, such as MOPO [27], and MOREL [?]. The second one is to apply pessimistic learning methods, e.g., conservative Q learning methods, when training with the learned environment model, such as the COMBO [?] and the H2O [?]. A key factor of model-based offline RL algorithms is the generalization ability of the learned environment model since the quality of the generated imaginary data largely determines the policy training. In this paper, we build a PE environment model, taking advantage of the generalization ability of the PE neural networks.

A.3 Permutation Equivariance RL Algorithms

A permutation equivariant (PE) function is a type of function that maintains the same output structure under the reordering of its inputs. Given a set of inputs, if we apply a permutation (rearrange the order of the inputs), a permutation equivariant function will produce a set of outputs that are permuted in the same way as the inputs. A permutation invariant (PI) function is a type of function that produces the same output regardless of the order of its inputs. In other words, the output of the function is unchanged under any permutation of the input elements. The PE/PI functions can be implemented by many structures, such as the DeepSet [28], and a set of transformers [?], and have been used in many areas, such as RL, computer visions, and natural language processings [?]. However, most of the previous works employ PE/PI neural networks in modeling policies or value functions in RL algorithms [???]. For example, [?] uses the Deep Set to model the policies of the UAVs, taking advantage of the variable input dimensions and PI property of the Deep Set, and the policies are trained in a simulated

environment implemented by the OpenAI Gym interface that can be freely interacted with. [?] leverages the Deep Set and attention-based neural networks to encode the neighborhood information in the quadrotors' policies and value functions, and the algorithm is trained in a simulated environment that can be freely interacted with. In this paper, we utilize them to model the environment for the agent in RL algorithms, additionally taking advantage of their generalization abilities. Besides, we also highlight that

We are the first to prove the permutation equivariant/invariant (PE/PI) properties of the advertiser's transition rule and the reward function under a typical industrial auction mechanism that involves multiple stages (Proposition 5.1), which provides the basis for the PE environment model design. We are the first to both theoretically (E.q. (6)) and empirically (Table 2 and Table 3) demonstrate the generalization superiority of PE/PI neural networks in modeling the environment in the auto-bidding field.

B NOTATIONS AND DEFINITIONS

Notations: $\mathbb{N}_+$ and $\mathbb{R}_+$ represent the positive integer set and the positive real scalar set, respectively; $\cdot^\top$ represents the transpose operator; $[N]$ denotes the positive integer set containing 1 to N , where $N \in \mathbb{N}_+$; $[\cdot]_+ \triangleq \max\{\cdot, 0\}$; $|\mathcal{D}|$ represents the number of elements in the set $\mathcal{D}$; $\circ$ is the function composition operation; $\|\cdot\|_1$ denotes the 1-norm; $\cup$ denotes the union operator between sets, and $\setminus$ denotes the subtraction operator between sets;

We next give some definitions that will be used in later proves. Based on the permutation operator, we can define the *orbit averaging* operator as follows.

Definition B.1 (Orbit Averaging). An orbit averaging Q is an operator acting on functions $f : \mathcal{X} \rightarrow \mathcal{Y}$, i.e.,

$$Qf = \frac{1}{N!} \sum_{\rho \in \Omega_N} \rho^{-1} \circ f \circ \rho, \quad (18)$$

where $\mathcal{X}$ and $\mathcal{Y}$ are the spaces of N -dimensional ordered vectors.

Moreover, as we need to analyze the generalization bound for the design environment model that is measured by the r -covering number based on the l_∞ -norm [4], we formally define them as follows.

Definition B.2 (l_∞ -Norm). The l_∞ norm between two functions $f, g : \mathcal{X} \rightarrow \mathcal{Y}$ is defined as $\|f - g\|_\infty \triangleq \max_{\mathbf{x} \in \mathcal{X}} \|f(\mathbf{x}) - g(\mathbf{x})\|_1$.

Definition B.3 (R-covering Number). Given a function set $\mathcal{F}$, the function set $\mathcal{F}'$ is said to r -covers $\mathcal{F}$ under l_∞ norm if for any function $f \in \mathcal{F}$, there exists $f' \in \mathcal{F}'$, such that $l_\infty(f, f') \leq r$. The r -covering number of a function set $\mathcal{F}$, denoted as $N_\infty(\mathcal{F}, r)$, is the cardinality of the smallest function set that r -covers $\mathcal{F}$ under l_∞ norm.

C PROOF OF LEMMAS

C.1 Proof of Lemma D.2

LEMMA C.1. *The orbit averaging Q can make any function $f : \mathcal{X} \rightarrow \mathcal{Y}$ a PE function, i.e., $\rho(Qf(\mathbf{x})) = Qf(\rho\mathbf{x})$, for any $\mathbf{x} \in \mathcal{X}$. Moreover, if f is a PE function, then $Qf = f$.*

PROOF. Following the definition of the orbit averaging in Definition B.1, we have:

$$\begin{aligned} Qf(\rho\mathbf{x}) &= \frac{1}{N!} \sum_{\rho_1 \in \Omega_N} \rho_1^{-1} f(\rho_1 \rho \mathbf{x}) \\ &= \rho \left(\frac{1}{N!} \sum_{\rho_1 \in \Omega_N} (\rho_1 \rho)^{-1} f(\rho_1 \rho \mathbf{x}) \right) \\ &= \rho(Qf(\mathbf{x})). \end{aligned} \quad (19)$$

Note that $\rho_1 \rho$ is also a permutation operator in Ω_N , and when iterating ρ_1 through Ω_N , $\rho_1 \rho$ will also iterate through Ω_N without repetition. Besides, if f is already a PE function, we have:

$$Qf(\mathbf{x}) = \frac{1}{N!} \sum_{\rho \in \Omega_N} \rho^{-1} f(\rho \mathbf{x}) = \frac{1}{N!} \sum_{\rho \in \Omega_N} \rho^{-1} \rho f(\mathbf{x}) = f(\mathbf{x}). \quad (20)$$

This completes the proof. $\square$

C.2 Lemmas on Permutation Operators

Recall that the permutation operator ρ can swap elements in an ordered vector $\mathbf{a} \in \mathbb{R}^N$, i.e.,

$$\rho \mathbf{a} = \rho(a_1, a_2, \dots, a_N) = (a_{\rho^{-1}(1)}, a_{\rho^{-1}(2)}, \dots, a_{\rho^{-1}(N)}). \quad (21)$$

As a matrix $\mathbf{A} \in \mathbb{R}^{N \times N}$ can be viewed as an order vector composed of the row vectors $\mathbf{a}_{r,j} \in \mathbb{R}^{1 \times N}$, $i, j \in [N]$, i.e., $\mathbf{A} = (\mathbf{a}_{r,1:N}^\top)$, we can define the *row permutation operator* ρ_r on $\mathbf{A}$ as swapping its row vectors following ρ , i.e.,

$$\rho_r \mathbf{A} \triangleq \rho(\mathbf{a}_{r,1:N}^\top) = \begin{bmatrix} a_{1\rho^{-1}(1)} & a_{1\rho^{-1}(2)} & \cdots & a_{1\rho^{-1}(N)} \\ a_{2\rho^{-1}(1)} & a_{2\rho^{-1}(2)} & \cdots & a_{2\rho^{-1}(N)} \\ \vdots & \vdots & \ddots & \vdots \\ a_{N\rho^{-1}(1)} & a_{N\rho^{-1}(2)} & \cdots & a_{N\rho^{-1}(N)} \end{bmatrix}, \quad (22)$$

where a_{ij} denotes the element in the i -th row and the j -th column of $\mathbf{A}$. Similarly, we can define the *column permutation operator* ρ_c on $\mathbf{A}$ as swapping its column vectors $\mathbf{a}_{c,i} \in \mathbb{R}^{N \times 1}$ following ρ , i.e.,

$$\mathbf{A} \rho_c \triangleq \rho(\mathbf{a}_{c,1:N})^\top = \begin{bmatrix} a_{\rho^{-1}(1)1} & a_{\rho^{-1}(1)2} & \cdots & a_{\rho^{-1}(1)N} \\ a_{\rho^{-1}(2)1} & a_{\rho^{-1}(2)2} & \cdots & a_{\rho^{-1}(2)N} \\ \vdots & \vdots & \ddots & \vdots \\ a_{\rho^{-1}(N)1} & a_{\rho^{-1}(N)2} & \cdots & a_{\rho^{-1}(N)N} \end{bmatrix}. \quad (23)$$

Note that ρ_c and ρ_r are usually implemented by the **Permutation Matrix**, where every row and column has exactly a coefficient 1 and the rest coefficients are all 0's. Hence, we view ρ_r and ρ_c as matrices in the following. We next provide some lemmas that will be used later.

LEMMA C.2. *It holds that $\rho_c^\top \rho_c = \mathbf{I}$ and $\rho_r^\top \rho_r = \mathbf{I}$.*

PROOF. There are many ways to prove this well-known lemma. A quick way to prove it is that: ρ_c and ρ_r both belong to the orthogonal matrix whose inverse is just its transpose. $\square$

LEMMA C.3. *It holds that for any square matrix $\mathbf{A}$, we have $\text{diag}(\rho_c^\top \mathbf{A} \rho_c) = \rho \text{diag}(\mathbf{A})$, and $\text{diag}(\rho_r \mathbf{A} \rho_r^\top) = \rho \text{diag}(\mathbf{A})$.*

PROOF. We first investigate what operations does $\rho_c^\top$ do. Specifically, from Lemma C.2, we have:

$$\rho_c^\top \rho_c = \rho_c^\top \mathbf{I} \rho_c = \rho_c^\top \begin{bmatrix} \mathbf{1}_{\rho(1)}^\top \\ \mathbf{1}_{\rho(2)}^\top \\ \vdots \\ \mathbf{1}_{\rho(N)}^\top \end{bmatrix} = \mathbf{I} = \begin{bmatrix} \mathbf{1}_{\rho^{-1}(\rho(1))}^\top \\ \mathbf{1}_{\rho^{-1}(\rho(2))}^\top \\ \vdots \\ \mathbf{1}_{\rho^{-1}(\rho(N))}^\top \end{bmatrix}, \quad (24)$$

where $\mathbf{1}_{\rho(i)} \in \mathbb{R}^{N \times 1}$ refers to the vector with one element 1 at position $\rho(i)$ and zero elements at other positions. Hence, we can know that $\rho_c^\top$ swaps the row vectors of the matrix according to ρ . Then, we have

$$\begin{aligned} \text{diag}(\rho_c^\top \mathbf{A} \rho_c) &= \text{diag} \left(\rho_c^\top \begin{bmatrix} a_{1\rho^{-1}(1)} & a_{1\rho^{-1}(2)} & \cdots & a_{1\rho^{-1}(N)} \\ a_{2\rho^{-1}(1)} & a_{2\rho^{-1}(2)} & \cdots & a_{2\rho^{-1}(N)} \\ \vdots & \vdots & \ddots & \vdots \\ a_{N\rho^{-1}(1)} & a_{N\rho^{-1}(2)} & \cdots & a_{N\rho^{-1}(N)} \end{bmatrix} \right) \\ &= (a_{\rho^{-1}(1)\rho^{-1}(1)}, a_{\rho^{-1}(2)\rho^{-1}(2)}, \dots, a_{\rho^{-1}(N)\rho^{-1}(N)}) \\ &= \rho \text{diag}(\mathbf{A}). \end{aligned} \quad (25)$$

Now, we prove $\text{diag}(\rho_c^\top \mathbf{A} \rho_c) = \rho \text{diag}(\mathbf{A})$, and $\text{diag}(\rho_r \mathbf{A} \rho_r^\top) = \rho \text{diag}(\mathbf{A})$ can be proved in the same way. $\square$

C.3 Other Lemmas

LEMMA C.4 (LEMMA B.2 IN [4]). *If $|l(\cdot)| \leq c$ for constant $c > 0$ and $\forall f, f' \in \mathcal{F}$, $|l(f, u) - l(f', u)| \leq K \|f - f'\|_\infty$, then we have*

$$\Delta \leq 2 \inf_{r>0} \left\{ \sqrt{\frac{2 \ln N_\infty(\mathcal{M}', r)}{|S|}} + Kr \right\} + C_{\delta, S}, \quad (26)$$

where $\Delta \triangleq \mathbb{E}_{u \sim \mathcal{D}} [l(f, u)] - \frac{1}{m} \sum_{u \in S} l(f, u)$, and $C_{\delta, S} \triangleq 4 \sqrt{\frac{2 \ln(4/\delta)}{|S|}}$. Here $\mathcal{D}$ and S represent the real distribution of u and the training set of u , and $m \triangleq |S|$.

PROOF. See the proof of Lemma B.2 in [4]. $\square$

LEMMA C.5 (LEMMA B.6 IN [4]). *For function class $\mathcal{M}$ and orbit averaging operator Q , if $\forall M_1, M_2 \in \mathcal{M}$, we have $l_\infty(QM_1, QM_2) \leq l_\infty(M_1, M_2)$, then $N_\infty(Q\mathcal{M}, r) \leq N_\infty(\mathcal{M}, r)$ for any $r > 0$.*

PROOF. For any $r > 0$, denote $\mathcal{M}_r$ as the smallest r -covering set that covers $\mathcal{M}$ with size $N_\infty(\mathcal{M}, r)$. For any $M \in \mathcal{M}$, let $M_r \in \mathcal{M}_r$ be the function that r -covers M . We have $l_\infty(QM_r, QM) \leq l_\infty(M_r, M) \leq r$. Therefore, QM_r is a r -covering set of $Q\mathcal{M}$, and we have $N_\infty(Q\mathcal{M}, r) \leq |QM_r| \leq |\mathcal{M}_r| = N_\infty(\mathcal{M}, r)$. $\square$

LEMMA C.6 (LEMMA 4.3 IN [17]). *Let the reward function of two environment models M and $\hat{M}$ be the same, denoted as r , and let their transition rules be P and $\hat{P}$, respectively. Then, the performance difference of a policy π in two environment models satisfies:*

$$\eta_{\hat{M}}(\pi) - \eta_M(\pi) = \gamma \mathbb{E}_{(s,a) \sim d_P^\pi} [G_M^\pi(s, a)], \quad (27)$$

where d_P^π represents the state-action pair distributions under policy π in $\hat{M}$, and $G_M^\pi(s, a) = \sum_{s'} (\hat{P}(s, a) - P(s, a)) V_M^\pi(s')$.

PROOF. See proof of the Lemma 4.3 in [17]. $\square$

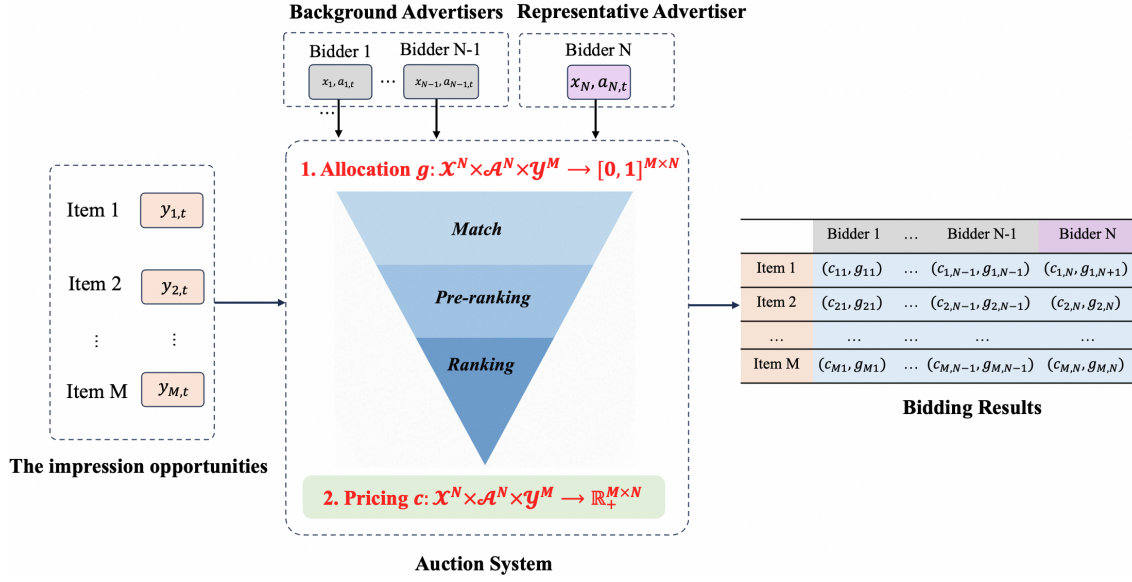


Figure 4: An illustration of the bidding process in an industrial advertising system between time step t and $t + 1$. Specifically, there are $N - 1$ background advertisers and one representative advertiser bidding for M impression opportunities, where $x_i \in \mathcal{X}$ denotes the contextual feature of advertiser i , $a_{i,t}$ is the bid of advertiser i , and $y_i \in \mathcal{Y}$ denotes the contextual feature of the j -th impression opportunity, $j \in [M]$. The auction system can be viewed as two mechanisms, including the allocation mechanism $g: \mathcal{X}^N \times \mathcal{A}^N \times \mathcal{Y}^M \rightarrow [0, 1]^{M \times N}$, and the pricing mechanism $c: \mathcal{X}^N \times \mathcal{A}^N \times \mathcal{Y}^M \rightarrow \mathbb{R}_+^{M \times N}$. The allocation mechanism g determines the distribution of the impression opportunities among the advertisers, while the pricing mechanism c decides how much each advertiser should pay.

D PROOF OF THEOREMS

THEOREM D.1 (BETTER GENERALIZATION ABILITY OF THE PE ENVIRONMENT MODEL). *For any environment model $\hat{M}'$, making it satisfy (4) can reduce the upper bound of the gap $\Delta_G(\hat{M}')$, i.e.,*

$$\text{Upper Bound of } \Delta_G(\hat{M}) \leq \text{Upper Bound of } \Delta_G(\hat{M}'), \quad (28)$$

where $\hat{M}$ denotes the environment model $\hat{M}'$ after making it satisfy (4).

PROOF. For the sake of explanation, we express the outputs of the PE environment model $\hat{M}$ as $\hat{\mathbf{u}}_t \triangleq \hat{\mathbf{u}}_{0:N,t}$ in this subsection, where $\hat{\mathbf{u}}_{i,t} = [\hat{s}_{i,t}, \hat{r}_{i,t}]^\top$, and the design in (4) can be equivalently written as:

$$\rho \hat{\mathbf{u}}_t = \hat{M}(\rho \mathbf{s}_t, \rho \mathbf{a}_t). \quad (29)$$

Consider a random environment model $\hat{M}'$ without the design of (29). A common way to make $\hat{M}'$ satisfy (29) is to apply the orbit averaging $\mathcal{Q}$ (as defined in Definition B.1) to it, i.e.,

$$\mathcal{Q}\hat{M}' = \frac{1}{N!} \sum_{\rho \in \Omega_N} \rho^{-1} \circ \hat{M}' \circ \rho. \quad (30)$$

This is because the orbit averaging $\mathcal{Q}$ can make any function a PE function. Formally, we have:

LEMMA D.2. *The orbit averaging $\mathcal{Q}$ can make any function $f: \mathcal{X} \rightarrow \mathcal{Y}$ a PE function, i.e., $\rho(\mathcal{Q}f(\mathbf{x})) = \mathcal{Q}f(\rho\mathbf{x})$, for any $\mathbf{x} \in \mathcal{X}$. Moreover, if f is a PE function, then $\mathcal{Q}f = f$.*

The proof is given in Appendix C.1. We consider the gap between the mean absolute error (MAE) of $\hat{M}'$ on the training dataset, denoted as $\mathcal{S}$, and that on the testing dataset, denoted as $\mathcal{D}$ ⁶, as the generalization performance metric of the environment model, i.e.,

$$\Delta_G(\hat{M}') \triangleq \mathbb{E}_{\mathcal{D}}[L(\hat{M}', \mathbf{s}_t, \mathbf{a}_t)] - \frac{1}{|\mathcal{S}|} \sum_{\mathcal{S}} L(\hat{M}', \mathbf{s}_t, \mathbf{a}_t), \quad (31)$$

where $L(\hat{M}', \mathbf{s}_t, \mathbf{a}_t) \triangleq \|\mathbf{u}_t - \hat{\mathbf{u}}_t\|_1$ is the MAE under $\mathbf{s}_t$ and $\mathbf{a}_t$, and $\mathbf{u}_t$ denotes the ground-truth. Next, we provide the upper bound of $\Delta_G(\hat{M}')$ in the following proposition.

PROPOSITION D.3 (GENERALIZATION BOUND). *Let $\mathcal{M}'$ be the function class of $\hat{M}'$, with probability at least $\delta > 0$, we have:*

$$\Delta_G(\hat{M}') \leq 2 \inf_{r>0} \left\{ \sqrt{\frac{2 \ln \mathcal{N}_\infty(\mathcal{M}', r)}{|\mathcal{S}|}} + r \right\} + C_{\delta, \mathcal{S}}, \quad (32)$$

where $C_{\delta, \mathcal{S}} \triangleq 4\sqrt{\frac{2 \ln(4/\delta)}{|\mathcal{S}|}}$, and $\mathcal{N}_\infty(\mathcal{M}', r)$ is the r -covering number (defined in Definition B.3) of $\mathcal{M}'$.

The proof is given in Appendix E.2. From (32) we know that the generalization bound $\Delta_G(\hat{M}')$ is positively correlated with the r -covering number of $\mathcal{M}'$, i.e., $\mathcal{N}_\infty(\mathcal{M}', r)$. After applying the orbit averaging to $\hat{M}'$, we get a new environment model $\hat{M}'' = \mathcal{Q}\hat{M}'$

⁶Theoretically, the testing dataset should be the real distribution of the state-action pairs $(\mathbf{s}_t, \mathbf{a}_t)$.

belonging to a new function class $\mathcal{M}'' \triangleq Q\mathcal{M}'$, where $Q\mathcal{M}'$ represents the function class obtained by applying the orbit averaging Q to every element in $\mathcal{M}'$. Notably, it can be proved that $\mathcal{M}''$ has a smaller r -covering number than $\mathcal{M}'$, which thereby decreases the generalization bound Δ_G .

PROPOSITION D.4 (SMALLER r -COVERING NUMBER). *Let $\mathcal{M}'' \triangleq Q\mathcal{M}'$. Then we have $N_\infty(\mathcal{M}'', r) \leq N_\infty(\mathcal{M}', r)$.*

Essentially, the orbit averaging Q is a non-expanding operator that can shrink the original function class $\mathcal{M}'$, making the new function class $\mathcal{M}''$ a smaller r -covering number. A formal proof is given in Appendix E.3. Now, we can claim that for a random environment model $\hat{\mathcal{M}}'$, making it satisfy (29) can reduce the upper bound of the gap between the MAE on the training dataset and the testing dataset, i.e.,

$$\text{Upper Bound of } \Delta_G(\hat{\mathcal{M}}'') \leq \text{Upper Bound of } \Delta_G(\hat{\mathcal{M}}') \quad (33)$$

can increase its generalization ability. $\square$

E PROOF OF PROPOSITIONS

E.1 Proof of Proposition 5.1

PROPOSITION E.1 (THE PE AND PI PROPERTIES OF THE POMDP, PROPOSITION 5.1 IN THE MAIN PAPER). *In the POMDP of the representation advertiser, the transition rule P is a PE function, i.e., $\rho s_{t+1} \sim P(\rho s_t, \rho a_t)$, and the reward R is a PI function, i.e., $R(\rho s_t, \rho a_t) = R(s_t, a_t)$, $\forall t, \rho$.*

PROOF. Essentially, the characteristics of the auction system endow the POMDP with the PE and PI properties. Consider the bidding process between time step t and $t + 1$, as shown in Fig. 4. Specifically, there are $N - 1$ background advertisers and one representative advertiser bidding for $M \in \mathbb{N}_+$ impression opportunities, where $x_i \in \mathcal{X}$ denotes the contextual feature of advertiser i , and $y_j \in \mathcal{Y}$ denotes the contextual feature of the j -th impression opportunity, $j \in [M]$. Here $\mathcal{X}$ and $\mathcal{Y}$ denote the contextual feature space of the advertiser and the impression opportunity, respectively. The auction system can be viewed as two mechanisms, including the allocation mechanism $g : \mathcal{X}^N \times \mathcal{A}^N \times \mathcal{Y}^M \rightarrow [0, 1]^{M \times N}$, and the pricing mechanism $c : \mathcal{X}^N \times \mathcal{A}^N \times \mathcal{Y}^M \rightarrow \mathbb{R}_+^{M \times N}$. The allocation mechanism g determines the distribution of the impression opportunities among the advertisers and is usually composed of cascaded stages, including match stage, pre-ranking stage, ranking stage, etc., and the pricing mechanism c decides how much each advertiser should pay. Let $G(\mathbf{x}, \mathbf{a}_t, \mathbf{y})$ and $C(\mathbf{x}, \mathbf{a}_t, \mathbf{y})$ be the outputs of the allocation mechanism g and the pricing mechanism c , respectively. Denote g_{ji} and c_{ji} as the element in the j -th row and the i -th column of G and C , respectively, meaning the allocation decision and the corresponding cost of impression opportunity j concerning advertiser i . Note that $\sum_i g_{ji} = 1$. An important characteristic of the allocation mechanism g is that it is permutation equivariant. Specifically, let the joint contextual feature of N advertisers and M impression opportunities be $\mathbf{x} \triangleq (x_{1:N})$ and $\mathbf{y} \triangleq (y_{1:M})$, respectively, and for any permutation operator ρ we have:

$$G(\rho \mathbf{x}, \rho \mathbf{a}_t, \mathbf{y}) = G(\mathbf{x}, \mathbf{a}_t, \mathbf{y}) \rho_c. \quad (34)$$

The reason why the allocation mechanism g is permutation equivariant is that the order of the advertiser sequence is not important

and cannot change the bidding results. Similarly, the pricing function c is also permutation equivariant, i.e.,

$$C(\rho \mathbf{x}, \rho \mathbf{a}_t, \mathbf{y}) = C(\mathbf{x}, \mathbf{a}_t, \mathbf{y}) \rho_c. \quad (35)$$

Let $\mathbf{V}(\mathbf{x}, \mathbf{y}) \in \mathbb{R}_+^{M \times N}$ be the value matrix, where the element in the j -th row and the i -th column $v_{ji} \in \mathbb{R}_+$ is the value of impression opportunity j with respect to advertiser i . We claim that the value matrix also satisfies the permutation equivariant property, i.e.,

$$\mathbf{V}(\rho \mathbf{x}, \mathbf{y}) = \mathbf{V}(\mathbf{x}, \mathbf{y}) \rho_c. \quad (36)$$

The reason is also that the order of the advertiser sequence is not important and cannot change the value evaluation of the impression opportunities. During the bidding process, there are only two key factors each advertiser i mainly focuses on, i.e., the value of the impression opportunities it can gain, $\sum_{j \in [M]} g_{ji} v_{ji}$, and the cost it should pay, $\sum_{j \in [M]} c_{ji}$. The former one is exactly the reward $r_{i,t}$ advertiser i receives, and the joint reward can be expressed as $\mathbf{r}_t(s_t, \mathbf{a}_t) = \text{diag}(G(\mathbf{x}, \mathbf{a}_t, \mathbf{y})^\top \mathbf{V}(\mathbf{x}, \mathbf{y}))$. Note that x_i is contained in the local state $s_{i,t}$. Hence, based on Lemma C.3 and the permutation equivariant property (34) and (36), we have

$$\begin{aligned} \mathbf{r}_t(\rho s_t, \rho \mathbf{a}_t) &= \text{diag}\left(G(\rho \mathbf{x}, \rho \mathbf{a}_t, \mathbf{y})^\top \mathbf{V}(\rho \mathbf{x}, \mathbf{y})\right) \\ &= \text{diag}\left((G(\mathbf{x}, \mathbf{a}_t, \mathbf{y}) \rho_c)^\top \mathbf{V}(\mathbf{x}, \mathbf{y}) \rho_c\right) \\ &= \text{diag}\left(\rho_c^\top G(\mathbf{x}, \mathbf{a}_t, \mathbf{y})^\top \mathbf{V}(\mathbf{x}, \mathbf{y}) \rho_c\right) \\ &= \rho \text{diag}\left(G(\mathbf{x}, \mathbf{a}_t, \mathbf{y})^\top \mathbf{V}(\mathbf{x}, \mathbf{y})\right) \\ &= \rho \mathbf{r}_t(s_t, \mathbf{a}_t). \end{aligned} \quad (37)$$

As the reward of the representative advertiser is the N -th element in $\mathbf{r}_t(s_t, \mathbf{a}_t)$, we have:

$$R(s_t, \mathbf{a}_t) = R(\rho s_t, \rho \mathbf{a}_t). \quad (38)$$

Let $\mathbf{J} \in \mathbb{R}^{M \times M}$ be the matrix with all 1 elements, and the joint cost can be expressed as $\mathbf{c}_t(s_t, \mathbf{a}_t) = \text{diag}(C(\mathbf{x}, \mathbf{a}_t, \mathbf{y})^\top \mathbf{J})$. Note that the change in state depends only on the cost spent between time step t and $t + 1$, i.e., $P(s_{t+1}|s_t, \mathbf{a}_t) = P(c_t(s_t, \mathbf{a}_t))$, where c_t is the corresponding cost during the state transition. Therefore, we have:

$$\begin{aligned} P\left(\rho s_{t+1} | \rho s_t, \rho \mathbf{a}_t\right) &= P\left(\mathbf{c}_t(\rho s_t, \rho \mathbf{a}_t)\right) \\ &= P\left(\text{diag}\left(C(\rho \mathbf{x}, \rho \mathbf{a}_t, \mathbf{y})^\top \mathbf{J}\right)\right) \\ &= P\left(\text{diag}\left(\rho_c^\top C(\mathbf{x}, \mathbf{a}_t, \mathbf{y})^\top \mathbf{J} \rho_c\right)\right) \\ &= P\left(\rho \text{diag}\left(C(\mathbf{x}, \mathbf{a}_t, \mathbf{y})^\top \mathbf{J}\right)\right) \\ &= P\left(\rho \mathbf{c}_t(s_t, \mathbf{a}_t)\right) \\ &= P\left(\mathbf{c}_t(s_t, \mathbf{a}_t)\right) \\ &= P\left(s_{t+1} | s_t, \mathbf{a}_t\right). \end{aligned} \quad (39)$$

Note that we use the properties $\mathbf{J} = \mathbf{J}\rho_c$ and $P(\rho_c(\mathbf{s}_t, \mathbf{a}_t)) = P(\mathbf{c}_t(\mathbf{s}_t, \mathbf{a}_t))$. So far, (38) and (39) have concluded the proof. $\square$

E.2 Proof of Proposition D.3

PROPOSITION E.2 (GENERALIZATION BOUND, PROPOSITION D.3 IN THE MAIN PAPER). *Let $\mathcal{M}'$ be the function class of $\hat{\mathcal{M}}'$, with probability at least δ , we have:*

$$\Delta_G \leq 2 \inf_{r>0} \left\{ \sqrt{\frac{2 \ln N_\infty(\mathcal{M}', r)}{|S|}} + r \right\} + C_{\delta, S}, \quad (40)$$

where $C_{\delta, S} \triangleq 4 \sqrt{\frac{2 \ln(4/\delta)}{|S|}}$, and $N_\infty(\mathcal{M}', r)$ is the r -covering number of $\mathcal{M}'$ defined in Definition B.3.

PROOF. We first show that: for any two functions $\hat{\mathcal{M}}_1$ and $\hat{\mathcal{M}}_2$, the loss function satisfies:

$$|L(\hat{\mathcal{M}}_1, \mathbf{s}_t, \mathbf{a}_t) - L(\hat{\mathcal{M}}_2, \mathbf{s}_t, \mathbf{a}_t)| \leq \|\hat{\mathcal{M}}_1 - \hat{\mathcal{M}}_2\|_\infty. \quad (41)$$

Without the loss of generality, let $L(\hat{\mathcal{M}}_1, \mathbf{s}_t, \mathbf{a}_t) > L(\hat{\mathcal{M}}_2, \mathbf{s}_t, \mathbf{a}_t)$. Then, we have:

$$\begin{aligned} & |L(\hat{\mathcal{M}}_1, \mathbf{s}_t, \mathbf{a}_t) - L(\hat{\mathcal{M}}_2, \mathbf{s}_t, \mathbf{a}_t)| \\ &= L(\hat{\mathcal{M}}_1, \mathbf{s}_t, \mathbf{a}_t) - L(\hat{\mathcal{M}}_2, \mathbf{s}_t, \mathbf{a}_t) \\ &= \|\mathbf{u}_t - \mathbf{u}_{1,t}\|_1 - \|\mathbf{u}_t - \mathbf{u}_{2,t}\|_2 \\ &= \sum_i |u_{i,t} - u_{i,1,t}| - \sum_i |u_{i,t} - u_{i,2,t}| \\ &= \sum_i |u_{i,t} - u_{i,1,t} + u_{i,2,t} - u_{i,2,t}| - \sum_i |u_{i,t} - u_{i,2,t}| \\ &\leq \sum_i \left[|u_{i,t} - u_{i,2,t}| + |u_{i,2,t} - u_{i,1,t}| - |u_{i,t} - u_{i,2,t}| \right] \\ &= \sum_i |u_{i,1,t} - u_{i,2,t}| \\ &= \|\mathbf{u}_{1,t} - \mathbf{u}_{2,t}\|_1 \\ &= \|\hat{\mathcal{M}}_1(\mathbf{s}_t, \mathbf{a}_t) - \hat{\mathcal{M}}_2(\mathbf{s}_t, \mathbf{a}_t)\|_1 \\ &\leq \max_{\mathbf{s}, \mathbf{a}} \|\hat{\mathcal{M}}_1(\mathbf{s}, \mathbf{a}) - \hat{\mathcal{M}}_2(\mathbf{s}, \mathbf{a})\|_1 \\ &= \|\hat{\mathcal{M}}_1 - \hat{\mathcal{M}}_2\|_\infty. \end{aligned} \quad (42)$$

Hence, we can leverage Lemma C.4 with $K = 1$. This finishes the proof. $\square$

E.3 Proof of Proposition D.4

PROPOSITION E.3 (SMALLER r -COVERING NUMBER). *Let $\mathcal{M}'' \triangleq \mathcal{Q}\mathcal{M}'$. Then we have $N_\infty(\mathcal{M}'', r) \leq N_\infty(\mathcal{M}', r)$.*

PROOF. Consider two random functions $\hat{\mathcal{M}}'_1, \hat{\mathcal{M}}'_2 \in \mathcal{M}'$, and let $\hat{\mathcal{M}}''_1 \triangleq \mathcal{Q}\hat{\mathcal{M}}'_1 \in \mathcal{M}''$ and $\hat{\mathcal{M}}''_2 \triangleq \mathcal{Q}\hat{\mathcal{M}}'_2 \in \mathcal{M}''$. With Lemma D.2, we only need to prove $l_\infty(\hat{\mathcal{M}}''_1, \hat{\mathcal{M}}''_2) \leq l_\infty(\hat{\mathcal{M}}'_1, \hat{\mathcal{M}}'_2)$. Specifically, we

have:

$$\begin{aligned} l_\infty(\hat{\mathcal{M}}'_1, \hat{\mathcal{M}}'_2) &= \max_{\mathbf{s}, \mathbf{a}} \|\hat{\mathcal{M}}'_1(\mathbf{s}, \mathbf{a}) - \hat{\mathcal{M}}'_2(\mathbf{s}, \mathbf{a})\|_1 \\ &= \max_{\mathbf{s}, \mathbf{a}} \|\mathcal{Q}\hat{\mathcal{M}}'_1(\mathbf{s}, \mathbf{a}) - \mathcal{Q}\hat{\mathcal{M}}'_2(\mathbf{s}, \mathbf{a})\|_1 \\ &= \max_{\mathbf{s}, \mathbf{a}} \left\| \frac{1}{N!} \sum_{\rho \in \Omega_N} \rho^{-1} \left(\hat{\mathcal{M}}'_1(\rho \mathbf{s}, \rho \mathbf{a}) - \hat{\mathcal{M}}'_2(\rho \mathbf{s}, \rho \mathbf{a}) \right) \right\|_1 \\ &\leq \max_{\mathbf{s}, \mathbf{a}} \frac{1}{N!} \sum_{\rho \in \Omega_N} \left\| \hat{\mathcal{M}}'_1(\rho \mathbf{s}, \rho \mathbf{a}) - \hat{\mathcal{M}}'_2(\rho \mathbf{s}, \rho \mathbf{a}) \right\|_1 \\ &= \frac{1}{N!} \sum_{\rho \in \Omega_N} \max_{\mathbf{s}, \mathbf{a}} \left\| \hat{\mathcal{M}}'_1(\rho \mathbf{s}, \rho \mathbf{a}) - \hat{\mathcal{M}}'_2(\rho \mathbf{s}, \rho \mathbf{a}) \right\|_1 \\ &= \frac{1}{N!} \sum_{\rho \in \Omega_N} \max_{\mathbf{s}, \mathbf{a}} \left\| \hat{\mathcal{M}}'_1(\mathbf{s}, \mathbf{a}) - \hat{\mathcal{M}}'_2(\mathbf{s}, \mathbf{a}) \right\|_1 \\ &= \frac{1}{N!} \sum_{\rho \in \Omega_N} l_\infty(\hat{\mathcal{M}}'_1, \hat{\mathcal{M}}'_2) \\ &= l_\infty(\hat{\mathcal{M}}'_1, \hat{\mathcal{M}}'_2). \end{aligned} \quad (43)$$

This concludes the proof. $\square$

E.4 Proof of Proposition 5.4

PROPOSITION E.4 (LOWER BOUND IMPROVEMENT, PROPOSITION 5.4 IN THE MAIN PAPER). *For any policy π , its performance in the online advertising system M , denoted as $\eta_M(\pi)$, is lower bounded by its performance in a fitted environment model $\hat{M}$ with the penalized predicted reward $\tilde{r}_t$, denoted as $\eta_{\hat{M}}(\pi)$, i.e., $\eta_M(\pi) \geq \eta_{\hat{M}}(\pi)$.*

PROOF. With Lemma C.6, we can get:

$$\begin{aligned} \eta_M(\pi) &= \mathbb{E}_{(\mathbf{s}, \mathbf{a}) \sim d_M^\pi} [R(\mathbf{s}, \mathbf{a}) - \gamma G_M^\pi(\mathbf{s}, \mathbf{a})] \\ &\geq \mathbb{E}_{(\mathbf{s}, \mathbf{a}) \sim d_M^\pi} [R(\mathbf{s}, \mathbf{a}) - \gamma |G_M^\pi(\mathbf{s}, \mathbf{a})|] \\ &\geq \mathbb{E}_{(\mathbf{s}, \mathbf{a}) \sim d_M^\pi} [R(\mathbf{s}, \mathbf{a}) - \lambda u(\mathbf{s}, \mathbf{a})] \\ &= \mathbb{E}_{(\mathbf{s}, \mathbf{a}) \sim d_M^\pi} [\tilde{r}_t] \\ &= \eta_{\hat{M}}(\pi). \end{aligned} \quad (44)$$

Note that we utilize a property [27] in the above derivation, i.e., there exists a function $u(\mathbf{s}, \mathbf{a})$ such that

$$u(\mathbf{s}, \mathbf{a}) \geq d_{\mathcal{F}} \triangleq \sup_{f \in \mathcal{F}} |\mathbb{E}_{\mathbf{s}' \sim \hat{p}} f(\mathbf{s}') - \mathbb{E}_{\mathbf{s}' \sim P} f(\mathbf{s}')| \geq |G_M^\pi|, \quad (45)$$

where $\mathcal{F}$ denotes the function class of the value function $V_M^\pi(\mathbf{s})$. Hence, (44) concludes the proof. $\square$

Received 20 February 2007; revised 12 March 2009; accepted 5 June 2009